JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY

Department of Computer Science & Engineering and Information Technology

Major Project Title: Kruxx

(A web application for summarizing educational content and generating personalized quizzes)

Enrollment No:	Name of Students:
21103105	Mukund Sarda
21103107	Vansh Gupta
21103110	Aryan Jolly

Name of Supervisor: Dr. Aastha Maheshwari

Course Name: Major Project Part-1 (15B19CI791)

Program: B. Tech. CSE.

Semester: 7th Semester

November - 2024

TABLE OF CONTENTS

Part No.	Description	Page No.
II	Declaration	4
III	Certificate	5
IV	Acknowledgement	6
V	Summary	7-8
VI	List of Figures	9
VII	List of Symbols and Acronyms	10
VIII	List of Tables	11
Chapter No.	**Topics**	**Page No.**
Chapter-1	**Introduction**	12-20
	1.1 General Introduction	12-13
	1.2 Problem Statement	14
	1.3 Significance of the Problem	15-16
	1.4 Empirical Study	17
	1.5 Brief Description of Solution Approach	18
	1.6 Comparison of Existing Approaches of the Problem Framed	19-20
Chapter-2	**Literature Summary**	21-23
	2.1 Summary of Papers Studied	21-22
	2.2 Integrated Summary of the Literature Studied	23
Chapter-3	**Requirement Analysis and Solution Approach**	24-34
	3.1 Overall Description of the Project	24

	3.2 Requirement Analysis	25-26
	3.3 Solution Approach	27-31
	3.4 Comparison with Existing Systems	32-34
Chapter-4	**Modelling and Implementation Details**	35-42
	4.1 Design Diagrams	35-37
	4.2 Implementation Details and Issues	38-39
	4.3 Risk Analysis and Mitigation	40-42
Chapter-5	**Testing**	43-49
	5.1 Test Plan	43
	5.2 Test Team Details	44
	5.3 Test Schedule	44
	5.4 Test Environment	45
	5.5 Component Decomposition and Type of Testing	45
	5.6 Test Cases	45-46
	5.7 Error and Exception Handling	46
	5.8 Limitations of the Solution	47-49
Chapter-6	**Findings, Conclusion and Future Work**	50-52
	6.1 Findings	50
	6.2 Conclusion	51
	6.3 Future work	52
	REFERENCES	53
	BIO DATA (RESUME)	54-56

(II)

DECLARATION

We, **Mukund Sarda**, **Vansh Gupta**, and **Aryan Jolly**, hereby declare that this submission is our own work and that, to the best of my knowledge and belief, it contains no material previously or written by another person nor material which has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgement has been made in the text.

Mukund Sarda	Vansh Gupta	Aryan Jolly
(21103105)	(21103107)	(21103110)

Place: JIIT, Noida, India

Date: 22 November, 2024

(III)

CERTIFICATE

This is to certify that the work titled **"Kruxx- A web application for summarizing educational content and generating personalized quizzes"** submitted by **Mukund Sarda**, **Vansh Gupta**, and **Aryan Jolly**, in partial fulfillment for the award of degree of B. Tech in Computer Science of Jaypee Institute of Information Technology, Noida has been carried out under my supervision. This work has not been submitted partially or wholly to another University or Institute for the award of this or other degree or diploma.

Dr. Aastha Maheshwari

(Assistant Professor- Senior Grade)

Date: 22 November, 2024

ACKNOWLEDGEMENT

We would like to express our sincere gratitude and heartfelt thanks to everyone who has contributed to the successful completion of this major project.

First and foremost, we extend our deepest appreciation to, Dr. Aastha Maheshwari, our project supervisor, for his invaluable guidance, unwavering support, and insightful feedback throughout the duration of this project. Their expertise and encouragement have been instrumental in shaping the project and enhancing its quality.

We would also like to thank, Jaypee Institute of Information Technology, for providing the necessary resources and conducive environment for conducting this project. The library and laboratory facilities have played a crucial role in the research and development process.

We would like to thank family and friends for their continuous encouragement and understanding during the challenging phases of this project. Their belief in our abilities has been a driving force behind our perseverance.

This project has been a significant learning experience, and we are truly grateful to have had the opportunity to work on it. Thank you all for being an integral part of this journey.

| --- | --- | --- |
| Mukund Sarda | Vansh Gupta | Aryan Jolly |
| (21103105) | (21103107) | (21103110) |

Date: 22 November, 2024

(V)

SUMMARY

The project titled **"Kruxx- A web application for summarizing educational content and generating personalized quizzes"** is a comprehensive solution designed to address the challenges faced by students and educators in today's fast-paced academic environment. By leveraging cutting-edge technologies such as Natural Language Processing (NLP), Optical Character Recognition (OCR), and advanced web development frameworks, the project provides an intelligent platform that simplifies the learning and assessment process.

Key Features of the Project

1. **Multi-Format Content Summarization**: Kruxx allows users to upload educational content in diverse formats, including:
    - Text documents (PDFs, DOCs).
    - Presentation files (PPTs).
    - Images (scanned notes or textbook pages).
    - Video links (e.g., YouTube lectures).
    - Audio files (recorded lectures or podcasts). Using advanced summarization models such as BART and BERT, the application condenses lengthy content into concise, easy-to-understand summaries while retaining critical information and context.
2. **Dynamic Quiz Generation**: The platform automatically generates quizzes from the summarized content, enabling users to test their comprehension. Features include:
    - Multiple-choice questions (MCQs) tailored to the uploaded material.
    - Options for varying difficulty levels based on user preferences.
    - Real-time evaluation and feedback to highlight strengths and areas for improvement.
3. **Performance Analytics and Feedback**: Kruxx provides users with a detailed analysis of their quiz results, offering insights into their performance and suggesting focus areas for further study. This feature enhances personalized learning and empowers users to improve systematically.

4. **User-Friendly Interface**: Designed with simplicity and accessibility in mind, the platform features:
    - A clean, responsive web interface compatible with desktops, tablets, and mobile devices.
    - Intuitive navigation and clear instructions for seamless user interaction.

Objective: The primary objective of Kruxx is to revolutionize how students and educators interact with educational content. The system achieves this by:

- Reducing the time required for studying large volumes of material.
- Enhancing knowledge retention through active learning with quizzes.
- Providing a versatile tool that caters to students, educators, and institutions alike.

| --- | --- | --- |
| Mukund Sarda | Vansh Gupta | Aryan Jolly |
| (21103105) | (21103107) | (21103110) |

Dr. Aastha Maheshwari

Date: 22 November, 2024

LIST OF FIGURES

Figure	Description	Page No.
Figure 3.1	Website UI	30
Figure 3.2	Website UI	31
Figure 3.3	Website UI	31
Figure 4.1	Use Case Diagram	35
Figure 4.2	Class Diagram	35
Figure 4.3	Control Flow Diagram	36
Figure 4.4	Sequence Diagram	37
Figure 4.5	Activity Diagram	37

LIST OF SYMBOLS AND ACRONYMS

Symbols	Meaning
BART	Bidirectional Auto-Regressive Transformers
BERT	Bidirectional Encoder Representations from Transformers
API	Application Programming Interface
OCR	Optical Character Recognition
NLP	Natural Language Processing
PDF	Portable Document Format
AI	Artificial Intelligence
HTML	Hypertext Markup Language
USP	Unique Selling Proposition

LIST OF TABLES

S. No.	Description	Page No.
1	Summarization	19
2	Quiz Generation	19
3	Unified Workflow	19
4	Summary of Papers Studied	21-22
5	Feature Comparison Table	33
6	Risk Identification	40
7	Risk Area Analysis	41
8	Mitigation Approaches	41-42
9	Test Plan	43
10	Test Team Details	44
11	Test Schedule	44
12	Test Environment	45
13	Component Decomposition and Type of Testing	45
14	Test Cases	45-46
15	Error and Exception Handling	46

CHAPTER- 1

INTRODUCTION

1.1) GENERAL INTRODUCTION

In today's fast-paced and information-rich world, students and educators face the ever-growing challenge of managing large volumes of educational content. Whether it's preparing for competitive exams, revising course material, or teaching students effectively, the task of processing, summarizing, and understanding information is often time-consuming and overwhelming.

Kruxx is a modern web-based solution that aims to bridge the gap between traditional learning methods and technological advancements. It combines cutting-edge techniques in Natural Language Processing (NLP), Optical Character Recognition (OCR), and web development to provide a platform that simplifies the way users interact with educational content. By automating processes such as summarizing study material and generating personalized quizzes, Kruxx enhances productivity, encourages active learning, and ensures better knowledge retention.

Why Kruxx?

Education today spans multiple media, including textbooks, research papers, presentations, video lectures, and even podcasts. Despite the availability of these resources, students often face:

1. **Information Overload**: The abundance of study materials leads to inefficiency in extracting key concepts and identifying critical areas for focus.
2. **Limited Time**: With tight deadlines for exams or assignments, students lack the time to process extensive content.
3. **Lack of Personalized Assessment**: Generic quizzes and assessments fail to address individual learning needs and weaknesses.

Educators, on the other hand, struggle to create personalized quizzes, analyse student performance, and adapt teaching strategies to suit diverse learners. Kruxx addresses these challenges by integrating summarization, quiz generation, and performance analytics into a unified platform.

Core Components

1. **Content Summarization**: Kruxx employs state-of-the-art NLP models like BART and BERT to generate summaries that retain essential information while eliminating redundant data. The

summarization module caters to both abstractive (rephrasing) and extractive (key sentence extraction) needs.

2. **Quiz Generation**: The platform dynamically creates quizzes based on summarized content, ensuring relevance and contextual alignment. Quizzes are generated with varying levels of difficulty to cater to users with diverse levels of understanding.
3. **OCR Integration**: For non-text inputs such as scanned notes or handwritten documents, Kruxx uses advanced OCR tools like EasyOCR and DocTR to extract text efficiently. This functionality makes the platform versatile and suitable for users who rely on physical study materials.
4. **Performance Analytics**: After quiz completion, Kruxx provides actionable feedback by analyzing performance trends. This helps users identify their strengths and weaknesses, allowing for personalized learning paths.

Kruxx is more than just a tool—it is a comprehensive learning assistant designed to simplify and enhance the educational experience. By combining automation, scalability, and advanced AI capabilities, Kruxx transforms the way users interact with educational content, fostering smarter study habits and improved academic performance. The platform's versatility and user-centric design make it a valuable resource for students, educators, and institutions alike.

1.2) PROBLEM STATEMENT

In today's educational landscape, students and professionals alike face a significant challenge in efficiently preparing for exams or skill assessments due to the overwhelming amount of study material available. This material comes in various formats—text, PDFs, video lectures, and audio recordings—making it hard for individuals to focus on the most relevant information. The problem is compounded by the following factors:

1. **Information Overload:** The sheer volume of data in textbooks, online resources, and lecture notes often leads to confusion, procrastination, and inefficient study practices.
2. **Lack of Personalization:** Most study aids do not cater to individual learning needs. Students have different levels of understanding, yet the available tools often provide generalized content.
3. **Limited Time:** Students often have limited time to prepare for exams, especially during last-minute revision phases. This necessitates a concise yet comprehensive way to review materials.
4. **Inadequate Self-Assessment Tools:** Assessing one's knowledge is crucial for effective preparation. However, many existing platforms fail to provide personalized and adaptive quizzes tailored to the user's study material.

Impact of Solving the Problem

The successful implementation of **Kruxx** will revolutionize the way students prepare for exams by addressing their pain points in summarization and assessment. It will significantly reduce study time, improve comprehension, and foster better learning outcomes, making education more accessible and efficient.

This project aligns with the broader goal of democratizing education by providing intelligent tools that cater to individual learning needs.

1.3) SIGNIFICANCE OF THE PROBLEM

The education landscape has undergone a massive transformation in recent years due to advancements in technology, the rise of online learning platforms, and the explosion of available educational resources. While these developments have democratized access to knowledge, they have also introduced a significant challenge: information overload. Students and educators alike are grappling with the sheer volume of content they need to process, comprehend, and retain within limited time frames.

The significance of the problem lies in addressing the following critical challenges:

1. **Managing Information Overload**

With the availability of diverse resources such as textbooks, research papers, online videos, and podcasts, students face the daunting task of:

- Filtering relevant information from voluminous content.
- Identifying key concepts and important areas of focus.
- Managing their time effectively to study, revise, and prepare for assessments.

2. **Lack of Summarization Tools for Diverse Formats**

While text summarization tools exist, they often cater to specific formats such as plain text or PDFs, leaving out other popular formats like:

- Scanned notes or handwritten documents.
- Video lectures from platforms like YouTube.
- Audio recordings of classroom sessions or presentations.

3. **Absence of Personalized Assessment**

Current assessment systems often fail to adapt to the individual learning needs of students. Key issues include:

- **Generic assessments:** Predefined quizzes or tests do not cater to the specific weaknesses or strengths of individual students.
- **One-size-fits-all approach:** Students with varying levels of understanding and preparation are assessed on the same parameters, leading to an inaccurate evaluation of their performance.

4. **Time Constraints in Exam Preparation**

The growing complexity of educational curricula, coupled with the limited time available for exam preparation, creates immense pressure on students. Key challenges include:

- Difficulty in organizing and prioritizing study materials.
- Inadequate time for revision due to lengthy, unstructured content.
- The mental stress of managing vast resources, which can hinder productivity and focus.

By automating processes like summarization and quiz generation, students can save significant time, allowing them to focus on understanding and mastering the material.

5. **Challenges for Educators**

Teachers and educators face their own set of challenges, including:

- **Time-consuming quiz creation:** Designing effective quizzes that align with the course material is a tedious and repetitive task.
- **Difficulty in evaluating student performance:** Manually analyzing quiz results and providing feedback is inefficient, especially for large classes.
- **Lack of tools to summarize teaching materials:** Educators often need to condense textbooks, research articles, or lecture notes into digestible formats for students.

These challenges prevent educators from focusing on what truly matters: teaching and mentoring students effectively.

6. **Inefficiencies in Traditional Learning Methods**

Traditional learning methods often rely on manual processes such as:

- Reading and highlighting key points from textbooks or notes.
- Writing summaries or creating flashcards for revision.
- Designing and solving practice questions for self-assessment.

These methods are not only time-consuming but also prone to errors, inconsistencies, and biases. Moreover, they fail to leverage the potential of modern technologies like Natural Language Processing (NLP) and automation, which can significantly enhance learning efficiency.

1.4) EMPIRICAL STUDY

The empirical study conducted as part of the **Kruxx** project aimed to evaluate real-world challenges faced by students and educators in preparing for exams and managing large volumes of educational content. The study provided critical insights into the limitations of existing tools, user expectations, and the feasibility of integrating advanced technologies like NLP and OCR into a unified platform.

The empirical study highlighted several critical pain points and opportunities for improvement in existing systems:

Challenges Faced by Students

1. **Information Overload**: 82% of students reported difficulty in managing the sheer volume of study material, especially during exam preparation. Students struggled to prioritize key concepts due to the lack of effective summarization tools.
2. **Time Constraints**: 78% of students expressed concern about the time required to manually summarize notes, textbooks, or video lectures. Many students admitted to skipping material due to insufficient time.
3. **Limited Assessment Tools**: While 70% of students used online quizzes for self-assessment, most found these quizzes to be generic and irrelevant to their specific needs.
4. **Diverse Input Formats**: Students often relied on handwritten notes, scanned documents, or video lectures, which were difficult to process using existing tools.

Challenges Faced by Educators

1. **Time-Consuming Quiz Creation**: 85% of educators found creating quizzes from scratch to be tedious and repetitive. Teachers expressed a desire for automated systems that could generate quizzes tailored to the content they teach.
2. **Performance Tracking**: Educators lacked tools to analyze student performance effectively and provide targeted feedback.

Existing Tool Limitations

1. **Narrow Focus**: Tools like SMMRY or Resoomer only supported plain text summarization, ignoring other formats like PDFs or video transcripts.
2. **Fragmented Solutions**: No single platform offered both summarization and quiz generation, requiring users to rely on multiple tools.

1.5) BRIEF DESCRIPTION OF SOLUTION APPROACH

The Kruxx system is designed as a modular web-based application, where each module is responsible for a specific function, ensuring scalability, maintainability, and ease of use. The architecture comprises the following key components:

1. **Frontend**: Built using HTML, CSS, and JavaScript for a responsive and user-friendly interface. Allows users to upload files, select summarization or quiz preferences, and view results.
2. **Backend**: Developed using Flask, a lightweight Python web framework, to handle user requests, process data, and generate outputs. Acts as the central hub connecting the frontend to various processing modules (e.g., summarization, OCR, and quiz generation).
3. **Processing Modules**: Utilizes advanced NLP models to process and summarize content. Creates dynamic quizzes based on the summarized material. Extracts text from images or scanned documents for further processing.
4. **APIs and External Tools**: Includes tools like the YouTube Transcript API for processing video content and libraries like EasyOCR for text extraction from non-text formats.

Advantages of the Solution Approach

1. **Automation**: Eliminates the need for manual summarization and quiz creation, saving significant time and effort for both students and educators.
2. **Scalability**: The modular architecture ensures the platform can handle large user bases and diverse inputs without performance degradation.
3. **Personalization**: Offers tailored summaries and quizzes based on individual user needs.
4. **Accessibility**: Supports a wide range of input formats, making it inclusive for users with varying resources and preferences.
5. **High Accuracy**: Leverages state-of-the-art NLP models and OCR tools to ensure precise summarization and content extraction.

1.6) COMPARISON OF EXISTING APPROACHES TO THE PROBLEM FRAMED

Kruxx overcomes the limitations of existing tools and manual methods by integrating advanced technologies into a single, comprehensive platform.

Summarization

Feature	**Existing Tools**	**Kruxx**
Input Format Support	Plain text	PDF, DOC, images, video, audio
Summarization Type	Extractive only	Extractive and Abstractive
Customization	Limited	Customizable length and style
NLP Model Integration	Basic	Advanced models (BART, BERT)
Multimedia Support	Not supported	Fully supported

Quiz Generation

Feature	**Existing Tools**	**Kruxx**
Automation	Manual input required	Fully automated from input content
Contextual Relevance	User-dependent	Ensures alignment with uploaded material
Performance Analytics	Limited or none	Detailed insights and recommendations
Time Efficiency	Time-consuming	Instant quiz generation

Unified Workflow

Feature	**Existing Approaches**	**Kruxx**
Workflow Integration	Fragmented	Unified platform for all features
User Interaction	Multiple tools needed	Single interface for all processes
Accessibility	Varies	Cross-platform and user-friendly

Why Kruxx is Better

1. **Time Efficiency**:
    - Automates both summarization and quiz creation, significantly reducing the time required for these tasks.
2. **Comprehensive Features**:
    - Supports a wide range of input formats, including text, images, videos, and audio, making it versatile and inclusive.
3. **Personalized Learning**:
    - Generates quizzes tailored to individual users, with insights into performance and suggestions for improvement.
4. **Advanced Technology Integration**:
    - Leverages state-of-the-art NLP models (BART and BERT) and OCR tools for high accuracy and contextual relevance.
5. **User-Centric Design**:
    - Combines a responsive, intuitive interface with powerful backend processing for seamless user experience.

CHAPTER- 2

LITERATURE SURVEY

2.1) SUMMARY OF PAPERS STUDIED

The following table summarizes the research papers reviewed during the development of **Kruxx**. Each paper contributed to understanding specific aspects of summarization, quiz generation, and the integration of NLP technologies into educational systems.

Author(s)	Title	Publish Date	Summary	Link
Lewis	BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension	2019	This paper introduces BART, a transformer-based model designed for abstractive summarization. It explains how BART combines bidirectional and autoregressive properties to handle long text inputs and generate concise summaries. The pre-training tasks discussed helped in fine-tuning the model for summarization tasks in this project.	Link
Devlin	BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding	2019	The authors discuss BERT, a bidirectional transformer model for NLP tasks. The paper details its ability to extract meaningful features from text, making it suitable for extractive summarization. This model's capacity to retain contextual information was instrumental in implementing extractive summarization for Kruxx.	Link
Zhang	Pegasus: Pre-training with Extracted Gap-sentences for Abstractive Summarization	2020	This paper explores Pegasus, a pre-trained NLP model for abstractive summarization. It introduces a unique pre-training objective that enhances the summarization of large documents. The insights from Pegasus were considered while evaluating the appropriateness of models like BART for Kruxx.	Link

Baek	What is Wrong with Scene Text Recognition Models and Data?	2019	The authors analyze challenges in OCR technology and propose techniques for improving text recognition in complex environments. Their findings informed the choice of OCR tools (EasyOCR and DocTR) for extracting text from handwritten notes and scanned documents in Kruxx.	Link
Lipton	A Critical Review of Recurrent Neural Networks for Sequence Learning	2015	This paper reviews the use of Recurrent Neural Networks (RNNs) for sequence-based tasks like summarization. While transformers like BERT and BART outperformed RNNs, this paper provided context on the evolution of sequence modeling and how modern approaches surpass traditional methods.	Link
Piech	Deep Knowledge Tracing	2015	The paper explores how machine learning can be used to model students' learning behavior and predict their performance. These ideas inspired the development of the performance analytics module in Kruxx, which provides targeted feedback based on quiz results.	Link
Guo	Attention Mechanisms for Neural Machine Translation	2017	The authors describe the use of attention mechanisms in deep learning models, which were instrumental in improving summarization tasks. These mechanisms directly influenced the effectiveness of BART and BERT in handling long text inputs for summarization in Kruxx.	Link
Tack	Generating Multiple-Choice Questions Using Pre-trained Language Models	2021	This paper discusses methods for generating multiple-choice questions (MCQs) using pre-trained language models. The insights from this research guided the design of the quiz generation module in Kruxx, ensuring high contextual relevance and quality of the generated quizzes.	Link

2.2) INTEGRATED SUMMARY OF LITERATURE STUDIED

The research papers reviewed provide a strong foundation for developing the **Kruxx** platform. The integrated insights from these papers are summarized below:

1. **Summarization Models**

The papers on **BART**, **BERT**, and **Pegasus** highlight the evolution of summarization technologies. While BERT excels in extractive summarization by identifying and retaining critical sentences, BART and Pegasus enable abstractive summarization by rephrasing content concisely. These insights guided the integration of both summarization techniques into Kruxx, giving users the flexibility to choose based on their needs.

2. **Optical Character Recognition (OCR)**

The findings from OCR-focused research, such as Baek et al.'s work, emphasized the importance of robustness in text recognition. By leveraging EasyOCR and DocTR, Kruxx incorporates state-of-the-art OCR capabilities, ensuring accurate text extraction from scanned documents and handwritten notes.

3. **Quiz Generation and Learning Analytics**

The ideas from Tack et al. and Piech et al. underscore the growing potential of AI in education. Pre-trained models for question generation ensure that quizzes are contextually relevant, while performance analytics derived from learning behavior enable personalized feedback. These concepts were directly applied in Kruxx to enhance quiz relevance and provide actionable insights.

4. **General NLP Advancements**

Papers like Guo et al.'s on attention mechanisms highlight how deep learning techniques such as transformers revolutionized text processing. This laid the groundwork for using advanced models like BART and BERT in Kruxx for efficient summarization and question generation.

CHAPTER- 3

REQUIREMENT ANALYSIS AND SOLUTION APPROACH

3.1) OVERALL DESCRIPTION OF THE PROJECT

Key Features

1. **Multi-Format Input Support**: Accepts text documents (PDF, DOC, TXT), scanned notes (images), video links (e.g., YouTube), and audio files. Converts multimedia inputs into processable text using Optical Character Recognition (OCR) and speech-to-text technologies.
2. **Advanced Summarization**: Generates concise summaries by rephrasing the input text while preserving meaning (powered by the BART model). Extracts the most relevant sentences directly from the input content (powered by the BERT model). Customizable options for summary length: short, medium, or long.
3. **Dynamic Quiz Generation**: Automatically generates multiple-choice questions (MCQs) based on the summarized material. Incorporates advanced NLP techniques to ensure contextual relevance. Customizable quiz parameters, such as the number of questions and difficulty levels.
4. **Performance Feedback and Analytics**: Provides detailed feedback on quiz performance, including scores, strengths, and areas for improvement. Suggests personalized study recommendations based on quiz results.
5. **User-Friendly Interface**: A clean, intuitive web interface designed for ease of use. Responsive design compatible with desktops, tablets, and smartphones.

Unique Selling Points (USPs)

- **All-in-One Platform**: Combines summarization, quiz generation, and analytics in a single application.
- **Advanced AI Models**: Leverages state-of-the-art transformer models to deliver superior performance.
- **Versatile Input Handling**: Processes text, images, videos, and audio seamlessly.
- **Customization**: Offers personalized settings for summaries and quizzes, ensuring user-specific outcomes.

3.2) REQUIREMENT ANALYSIS

Functional Requirements

1. **Content Upload**: Users should be able to upload educational content in various formats, including: PDFs, DOCs, and PPTs; Images (scanned notes or textbook pages); Video URLs (e.g., YouTube lectures); Audio recordings (e.g., podcasts or lectures). The system should validate file formats and provide appropriate error messages for unsupported formats.
2. **Content Summarization**: The system should generate concise summaries using: **Abstractive summarization** for rephrased, natural-sounding summaries; **Extractive summarization** for extracting key sentences from the input. Users should be able to select summary length (short, medium, long).
3. **Quiz Generation**: Automatically generate multiple-choice questions (MCQs) from the summarized content. Include distractor options for each question to increase quiz complexity.
4. **User Interface**: Provide a clean, intuitive interface for uploading content, viewing summaries, and taking quizzes. Ensure compatibility across devices (desktop, tablet, and mobile).
5. **Multi-User Support**: Allow multiple users to access the platform simultaneously without performance degradation. Provide user accounts with options for storing past summaries and quiz results.
6. **Integration with APIs**: Use APIs like the YouTube Transcript API to extract text from video lectures. Integrate OCR tools like EasyOCR for processing images and scanned documents.

Non-Functional Requirements

1. **Performance**: The system should process uploaded content and generate summaries within few seconds for small to medium-sized inputs (e.g., PDFs under 10 pages). Quiz generation should be completed within 3 seconds for standard content.
2. **Security**: Implement secure data storage using encryption to protect user data and educational content. Use HTTPS for secure communication between the client and server. Ensure role-based access control to prevent unauthorized data access.
3. **Usability**: The system should be easy to navigate for users with varying technical skills. Provide tooltips, guides, and error messages to assist users in operating the platform.

4. **Reliability**: Ensure the system is highly reliable with minimal downtime. Implement a robust error-handling mechanism to recover from unexpected failures.
5. **Portability**: The system should be deployable on multiple operating systems (Windows, macOS, Linux) and cloud platforms.
6. **Maintainability**: Use modular architecture to facilitate easy updates and maintenance. Provide detailed documentation for developers and administrators.

Technical Constraints

1. **NLP Models**:
    - BART and BERT models require significant computational resources for summarization tasks. The system must optimize processing to avoid delays.
    - Use pre-trained models to reduce development time and computational costs.
2. **OCR Tools**:
    - Tools like EasyOCR and DocTR may have limitations in recognizing highly distorted or low-resolution text. The system should preprocess images to improve OCR accuracy.
3. **Infrastructure**:
    - Ensure compatibility with commonly available hardware and software configurations.
    - Optimize the platform for cloud deployment to support scalability.

3.3) SOLUTION APPROACH

The solution approach for Kruxx is based on the integration of advanced technologies such as Natural Language Processing (NLP), Optical Character Recognition (OCR), and intelligent quiz generation into a modular and scalable web-based platform. The approach is designed to:

1. Automate time-intensive tasks like summarizing large volumes of content and creating quizzes.
2. Ensure compatibility with diverse input formats, including text, images, videos, and audio files.
3. Provide real-time feedback and analytics to improve user learning outcomes.
4. Maintain scalability and security to support a wide range of users, from individual learners to large institutions.

Architectural Design

The system architecture for Kruxx follows a **modular design** that separates various functionalities into independent components. This ensures scalability, maintainability, and ease of integration. The architecture comprises three main layers:

1. **Frontend Layer**:
    - Built using HTML, CSS, and JavaScript for a responsive and user-friendly interface.
    - Implements dynamic forms and features for file upload, summarization selection, quiz generation, and result visualization.
2. **Backend Layer**:
    - Developed using Flask, a lightweight Python framework, to manage user requests and orchestrate processing tasks.
    - Connects the frontend to core processing modules, including summarization, OCR, and quiz generation. Handles session management, data storage, and communication with external APIs.
3. **Data Processing Layer**:
    - Performs the heavy lifting, including text summarization, OCR-based text extraction, and quiz generation.

- Integrates pre-trained NLP models (e.g., BART, BERT) and OCR libraries (e.g., EasyOCR, DocTR).
- Provides APIs for processing multimedia content like video and audio files.

Step-by-Step Workflow

The solution approach involves the following key steps, ensuring seamless functionality from user interaction to output generation:

Step 1: User Input

- Users interact with the web application to upload study materials or provide URLs for videos or audio files.
- Supported input formats include:
    - **Text files**: PDF, DOC, and TXT.
    - **Images**: Scanned notes, handwritten content, or textbook pages.
    - **Videos**: YouTube links, which are processed to extract transcripts.
    - **Audio files**: Recorded lectures or podcasts, converted to text using speech-to-text libraries.

Step 2: Preprocessing

Before the core operations (summarization or quiz generation), the uploaded content is preprocessed to ensure compatibility and improve quality:

1. **Text Preprocessing**:
    - Cleaning raw text by removing unnecessary characters, special symbols, and formatting inconsistencies.
    - Tokenizing the text into smaller units for easier processing by NLP models.
2. **Image Processing**:
    - Enhancing image quality using libraries like PIL (Python Imaging Library) to improve OCR accuracy.
    - Extracting text from images using EasyOCR or DocTR.

3. **Video/Audio Transcription**:
    - Converting video or audio content into text using the YouTube Transcript API or speech-to-text tools like Google Speech-to-Text.

Step 3: Content Summarization

Once the input is preprocessed, the summarization module generates concise summaries of the content:

1. **Abstractive Summarization**:
    - Powered by BART (Bidirectional and Auto-Regressive Transformers).
    - Rephrases input text to create human-like summaries while preserving the core meaning.
    - Suitable for generating summaries that are creative and contextually rich.
2. **Extractive Summarization**:
    - Utilizes BERT (Bidirectional Encoder Representations from Transformers).
    - Identifies and extracts key sentences directly from the input text.
    - Ideal for preserving original phrasing and structure in the summary.

Users can select their preferred summarization type (abstractive or extractive) and specify the desired length (short, medium, or long).

Step 4: Quiz Generation

The quiz generation module dynamically creates multiple-choice questions (MCQs) from the summarized content. Key processes include:

1. **Keyword Extraction**:
    - Identifies important terms and phrases using NLP techniques like Named Entity Recognition (NER) and Term Frequency-Inverse Document Frequency (TF-IDF).
2. **Question Formation**:
    - Constructs questions based on extracted keywords, ensuring contextual relevance.
    - For example, converting a sentence like "The mitochondria are the powerhouse of the cell" into "What is the powerhouse of the cell?"

3. **Distractor Generation**:
    - Generates plausible but incorrect options using pre-trained word embeddings (e.g., GloVe, Word2Vec).
    - Ensures that distractors are contextually appropriate to maintain quiz quality.
4. **Customization**:
    - Users can specify quiz parameters such as the number of questions and difficulty level.

Step 5: User Interface and Output Delivery

- Summaries, quizzes, and performance analytics are displayed on a clean and intuitive interface.
- Users can download summaries or save quiz results for future reference.

Website UI

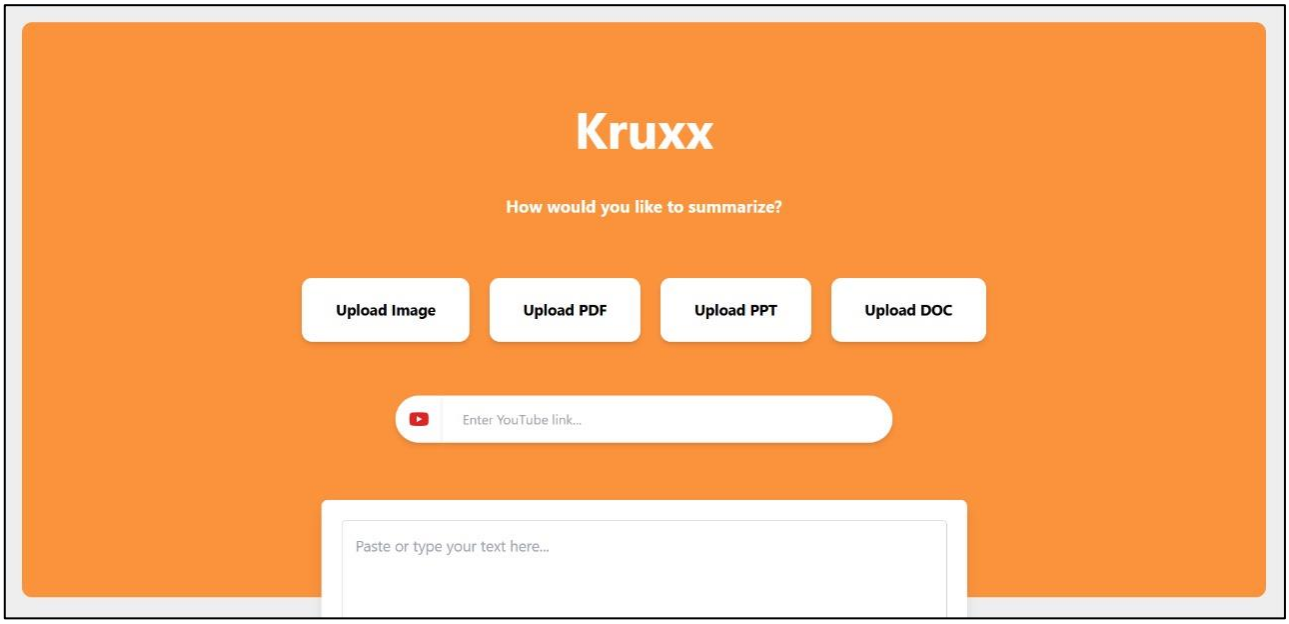

Fig 3.1

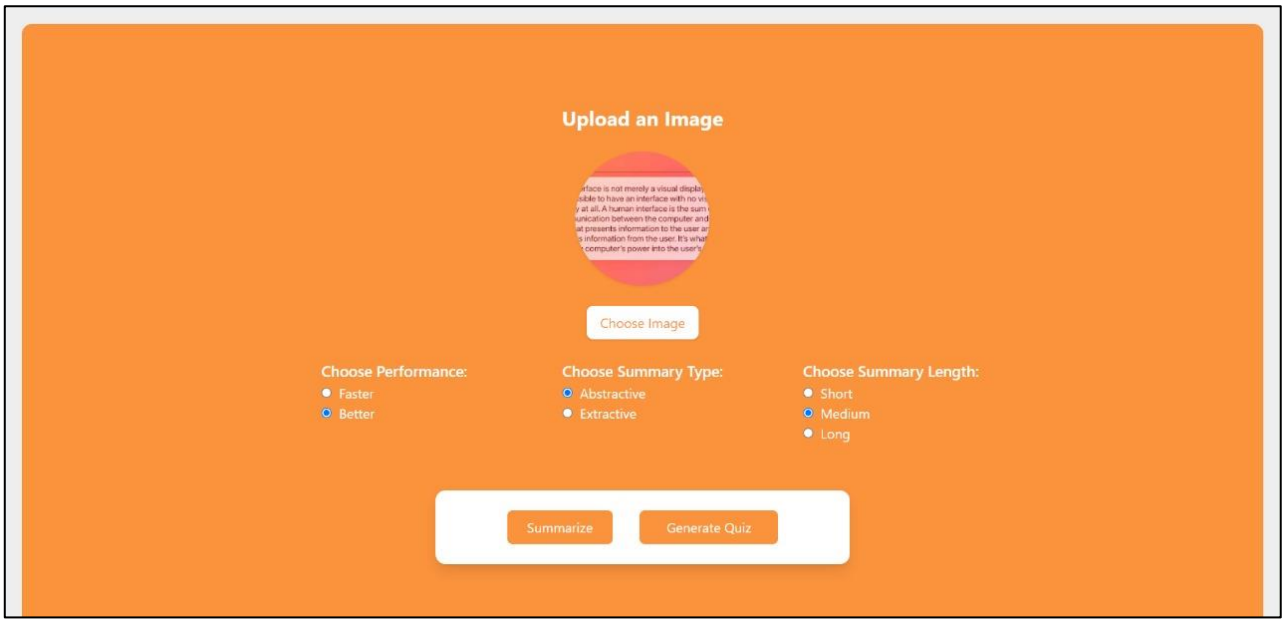
Upload an Image
Choose Image
Choose Performance:
Faster
Better
Choose Summary Type:
Abstractive
Extractive
Choose Summary Length:
Short
Medium
Long
Summarize
Generate Quiz

Fig 3.2

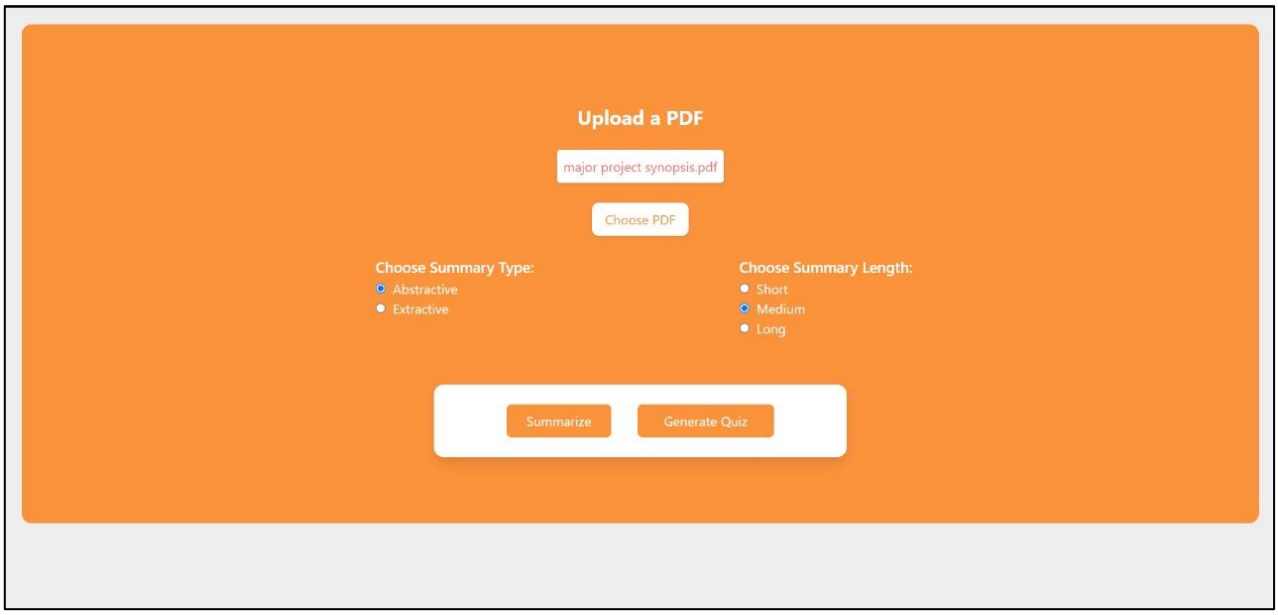
Upload a PDF
major project synopsis.pdf
Choose PDF
Choose Summary Type:
Abstractive
Extractive
Choose Summary Length:
Short
Medium
Long
Summarize
Generate Quiz

Fig 3.3

3.4) Comparison with Existing Systems

This section provides a detailed comparison of **Kruxx** with similar existing systems, analyzing its features, functionalities, and performance metrics. The goal is to highlight the unique advantages of **Kruxx** and demonstrate its superiority in solving the challenges of personalized exam preparation.

1. **Existing Systems**

Several platforms offer partial or similar functionalities to **Kruxx**, including content summarization tools, quiz generation systems, and learning management systems (LMS). Below are key competitors:

1. **QuillBot**:
    - **Functionality**: Primarily focuses on paraphrasing and summarization.
    - **Strengths**:
        - Effective at rephrasing text.
        - User-friendly and quick processing.
    - **Limitations**:
        - Does not support input formats like videos, images, or audio.
        - Lacks quiz generation and performance analytics.
2. **Kahoot!**:
    - **Functionality**: Primarily a quiz-based platform for educators and students.
    - **Strengths**:
        - Interactive and engaging quizzes.
        - Real-time multiplayer features.
    - **Limitations**:
        - No summarization capabilities.
        - Relies on manual quiz creation by educators.
3. **Edmodo**:
    - **Functionality**: Learning management system with classroom collaboration tools.
    - **Strengths**: Facilitates classroom communication and resource sharing.

- **Limitations**:
    - Lacks content summarization and automated quiz generation.
    - Limited to text-based inputs.

4. **SummarizeBot**:
    - **Functionality**: Provides automated summarization for text, PDFs, and web pages.
    - **Strengths**: Supports multiple input formats.
    - **Limitations**:
        - No quiz generation or performance tracking.
        - Summaries lack customization for length or type.

Feature Comparison Table

Feature	**Kruxx**	**QuillBot**	**Kahoot!**	**Edmodo**	**SummarizeBot**
Content Summarization	Yes (BERT/BART)	Yes	No	No	Yes
Customizable Summaries	Yes (Short/Med/Long)	Limited	No	No	Limited
Quiz Generation	Yes (MCQs)	No	Yes (Manual)	No	No
Input Formats	Text, PDFs, DOCs, PPTs, Images, Videos, Audio	Text only	None (Quiz Only)	Text only	Text, PDFs
Performance Analytics	Yes	No	Limited	Limited	No
Real-Time Collaboration	Planned	No	Yes	Yes	No
Multimedia Support	Yes	No	No	No	No

2. **Key Differentiators of Kruxx**

1. **Comprehensive Functionality**:
    - Combines summarization, quiz generation, and analytics in a single platform, eliminating the need for multiple tools.
2. **Wide Input Format Support**:
    - Handles diverse input formats, including text, images, videos, and audio, making it suitable for a variety of learning resources.
3. **Customizable Summaries**:
    - Offers both extractive (BERT) and abstractive (BART) summarizations with options for short, medium, and long summaries.
4. **Dynamic Quiz Generation**:
    - Automatically generates multiple-choice questions (MCQs) based on summarized content, including contextually relevant distractors.
5. **Personalized Feedback**:
    - Provides detailed analytics on quiz performance, highlighting strengths and areas for improvement.
6. **Scalability**:
    - Designed to handle concurrent user interactions with optimized backend architecture and caching mechanisms.
7. **Future-Proof Features**:
    - Plans to integrate multilingual support and real-time collaboration, further enhancing its appeal.

CHAPTER- 4

MODELLING AND IMPLEMENTATION DETAILS

4.1) DESIGN DIAGRAMS

1. Use Case Diagram

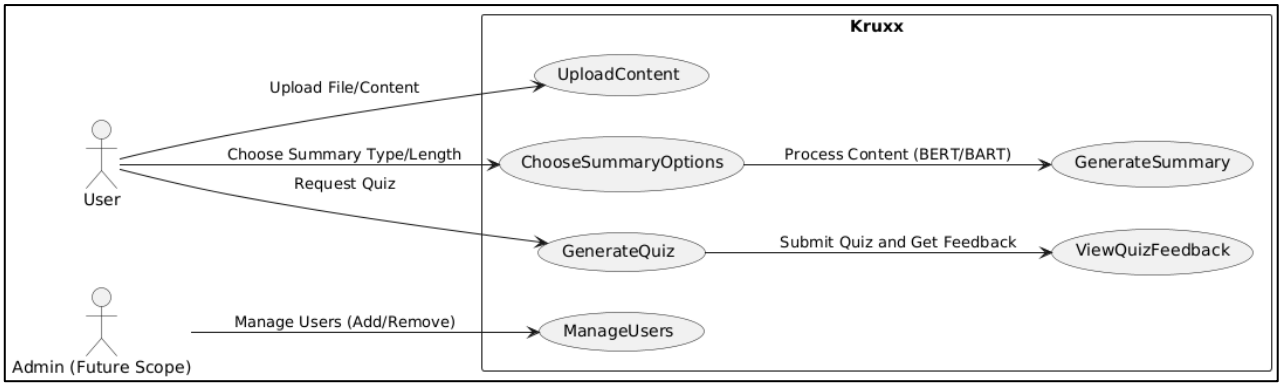

Fig 4.1

2. Class Diagram

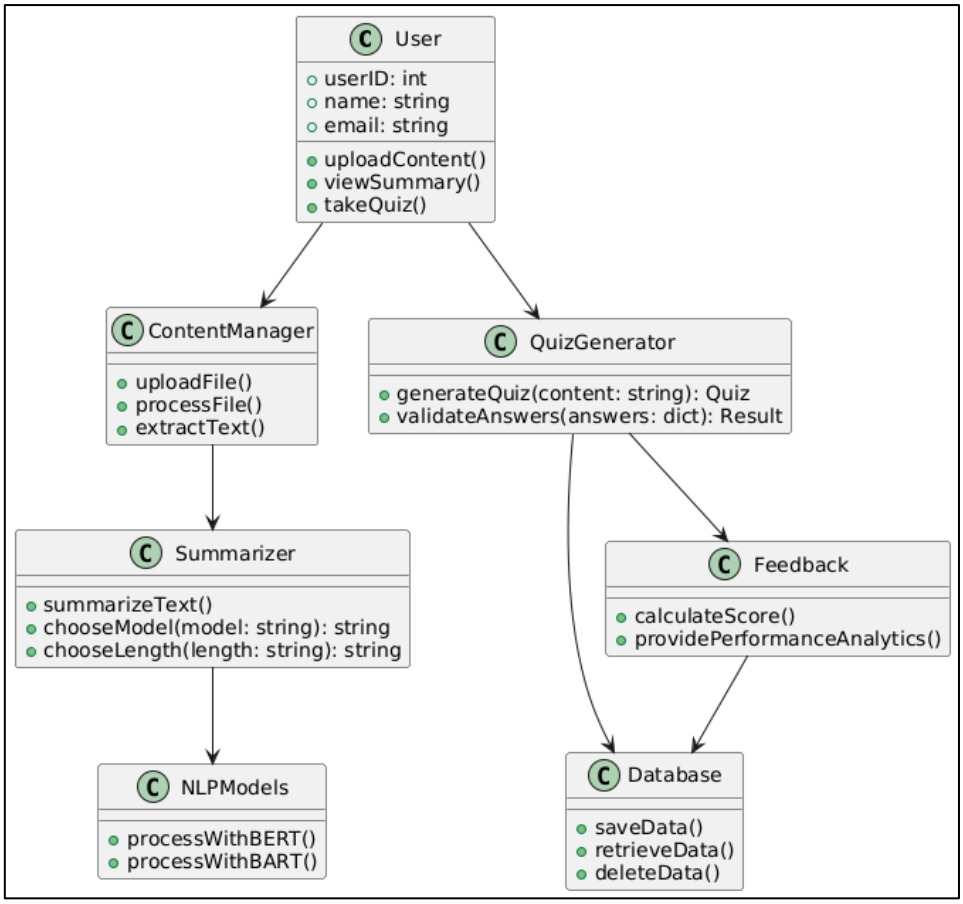

Fig 4.2

3. Control Flow Diagram

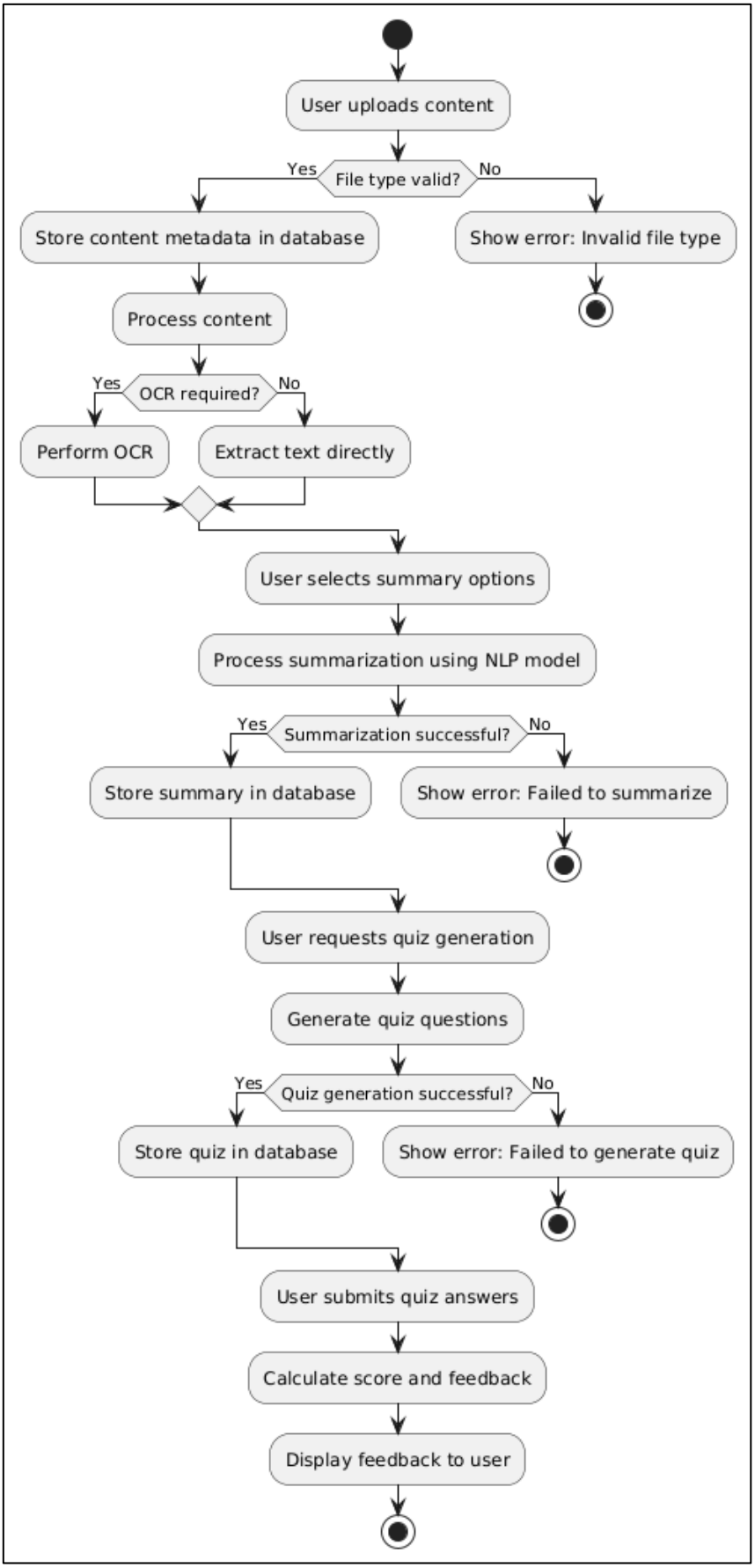

Fig 4.3

4. Sequence Diagram

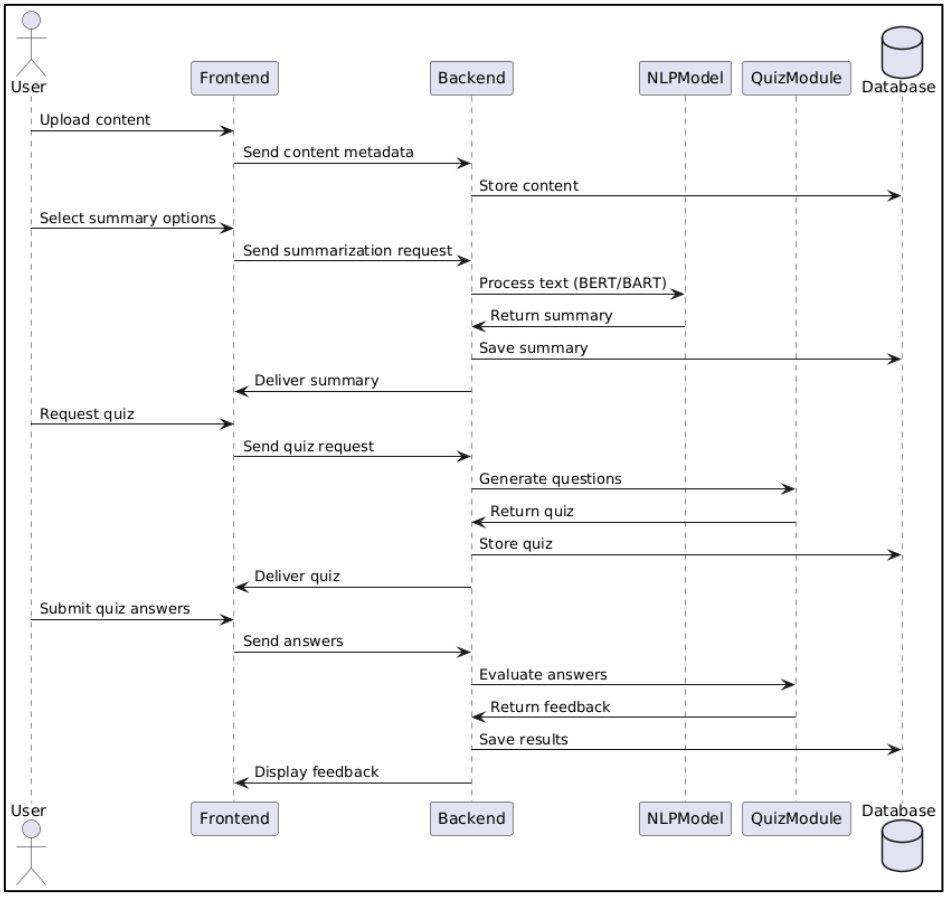

Fig 4.4

5. Activity Diagram

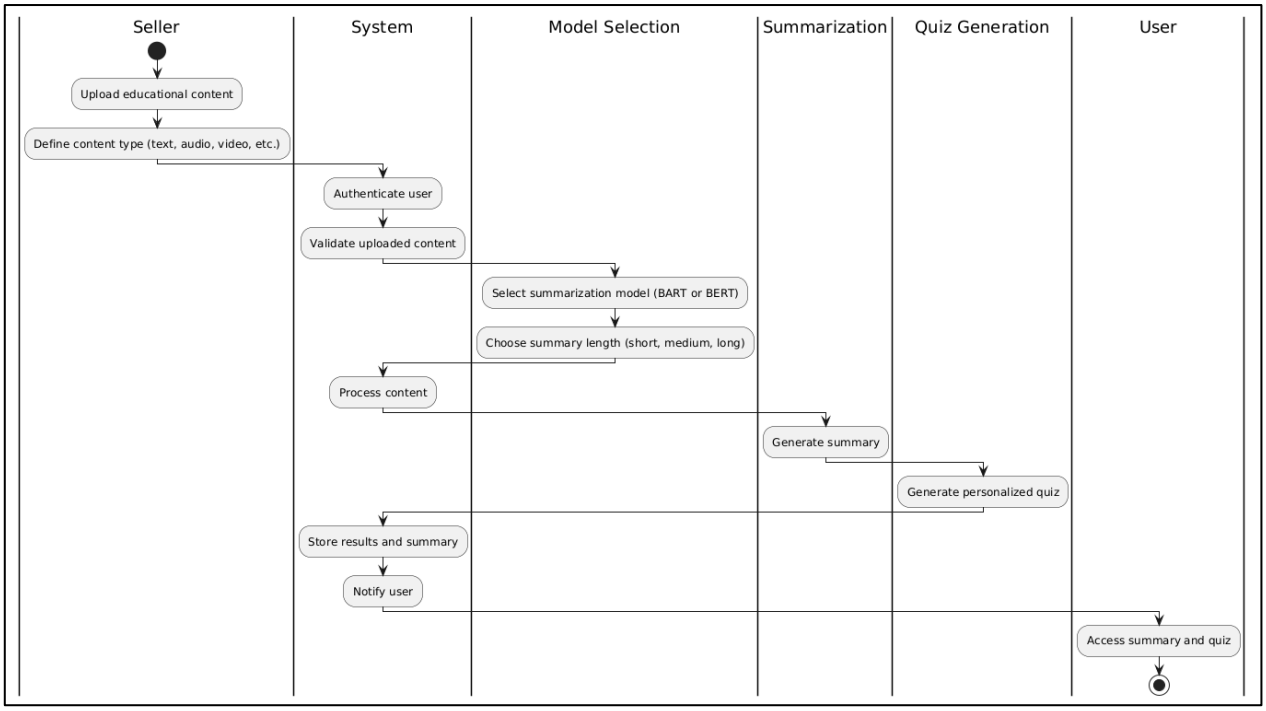

Fig 4.5

4.2) IMPLEMENTATION DETAILS AND ISSUES

1. **Backend Development**

Activities

1. Implement session management using Flask Sessions.
2. Uploaded content and file types. Summaries and quiz data.
3. Integrate NLP models (BERT and BART) for summarization.

Technologies

- **Flask**: A lightweight Python web framework for API and backend development.
- **Transformers Library**: For pre-trained NLP models like BERT and BART.

Deliverables

- A secure backend with user authentication.
- Support for uploading, processing, and storing content summaries and quiz data.

2. **Summarization and Quiz Generation Modules**

Activities

1. Summarization:
    - Integrate **BERT** for extractive summarization.
    - Use **BART** for abstractive summarization.
2. Quiz Generation:
    - Extract keywords using **TF-IDF** and **NER**.
    - Generate distractor options using **Word2Vec** embeddings.

Technologies

- **HuggingFace Transformers**: For NLP model implementation.
- **SpaCy**: For Named Entity Recognition (NER).
- **TF-IDF**: To calculate important keywords for quiz questions.

Deliverables

- A fully functional summarization system.
- Dynamic quiz generation with customizable difficulty levels.

3. **Frontend Development**

Activities

1. Design user-friendly interfaces for uploading files, selecting summarization options, and taking quizzes.
2. Create responsive forms for different file types (PDF, DOC, images, videos).
3. Display feedback and performance analytics using interactive charts.

Technologies

- **HTML/CSS**: For basic design and layout.
- **JavaScript**: To enable dynamic functionalities.
- **Bootstrap**: For responsive and clean UI components.

Deliverables

- A responsive frontend supporting multi-device usage.
- Dynamic pages for summaries, quizzes, and analytics.

4. **Issues Encountered**

1. **Model Performance on Large Inputs**: Both BERT and BART struggled with input sizes exceeding the token limit. **Solution**: Split large documents into manageable chunks.
2. **OCR Accuracy for Handwritten Notes**: Low accuracy with cursive or unclear handwriting. **Solution**: Preprocessing images to enhance clarity using libraries like **PIL**.
3. **Complexity of Dynamic Quiz Generation**: Generating contextually relevant distractors was challenging. **Solution**: Use **Word2Vec** to fetch semantically similar words.
4. **Performance Overhead**: High computation time during batch processing. **Solution**: Cache results and preload models into memory.

4.3) RISK ANALYSIS AND MITIGATION

4.3.1 **Risk Identification**

Risk ID	Classification	Description of Risk	Risk Area	Probability (P)	Impact (I)	Risk Exposure (P × I)
R1	Product Engineering - Security	Unauthorized access to user-uploaded content or quiz results.	Data Security	High (4)	High (5)	20
R2	Product Engineering - Scalability	Backend fails to handle concurrent user interactions during peak usage.	System Scalability	High (4)	High (5)	20
R3	Development Environment - Usability	Users find it difficult to navigate the platform or use advanced features.	User Experience	Medium (3)	High (4)	12
R4	Product Engineering - Model Accuracy	Summarization models produce inaccurate or irrelevant summaries.	Model Performance	Medium (3)	High (4)	12
R5	Product Engineering - Integration	API failures disrupt communication between frontend, backend, and database.	Backend/API Integration	Medium (3)	High (4)	12
R6	Code and Unit Test - OCR Performance	OCR fails to accurately extract text from low-quality handwritten images.	OCR Reliability	High (4)	Medium (3)	12

4.3.2 **Risk Area Analysis**

Risk Area	Number of Risk Statements	Weights (In + Out)	Total Weight
Data Security	1	9 + 9	18
System Scalability	1	9 + 6	15
User Experience	1	6 + 6	12
Model Performance	1	6 + 6	12
Backend/API Integration	1	3 + 3	6
OCR Reliability	1	6 + 3	9

4.3.3 **Mitigation Approaches**

Risk ID	Risk Statement	Risk Area	Mitigation Approach	Start Date	Completion Date	Additional Resources Needed
R1	Unauthorized access to user-uploaded content or quiz results.	Data Security	Implement AES-256 encryption and multi-factor authentication.	01/12/24	10/12/24	Encryption libraries, Authentication tools
R2	Backend fails to handle concurrent user interactions during peak usage.	System Scalability	Optimize backend queries, implement caching, and use a load balancer.	05/12/24	15/12/24	Redis/Memcached, Load balancing tools
R3	Users find it difficult to navigate the platform or use	User Experienc e	Conduct usability testing and redesign UI	03/12/24	20/12/24	Usability testing tools, Prototyping software

	advanced features.		based on feedback.			
R4	Summarization models produce inaccurate or irrelevant summaries.	Model Performance	Fine-tune models using domain-specific datasets; test summaries rigorously.	01/12/24	18/12/24	Fine-tuning datasets, Model evaluation tools
R5	API failures disrupt communication between frontend, backend, and database.	Backend/ API Integration	Conduct regular API testing and implement retries for critical endpoints.	08/12/24	22/12/24	Postman, Integration testing frameworks
R6	OCR fails to accurately extract text from low-quality handwritten images.	OCR Reliability	Preprocess images using filters to enhance quality before OCR.	12/12/24	25/12/24	Image preprocessing libraries (Pillow, OpenCV)

CHAPTER- 5

TESTING

5.1 Test Plan

Type of Test	Performance	Status	Explanation	Software Component
Functional Testing	Yes	Pass	Verifies file upload, summarization, and quiz generation.	Frontend, Backend, Database
Unit Testing	Yes	Pass	Validates individual modules like summarization and OCR.	Backend APIs
Integration Testing	Yes	Pass	Tests interactions between frontend, backend, and database.	Full-stack Integration
Performance Testing	Yes	Pass	Validates the speed of NLP processing and database queries.	Backend, NLP Models
Stress Testing	Yes	Pass	Simulates high traffic scenarios to check system robustness.	Backend, Database
Load Testing	Yes	Pass	Assesses system performance under typical user workloads.	Full Stack System
Volume Testing	Yes	Pass	Ensures database stability with large datasets.	Database
End-to-End Testing	Yes	Pass	Validates the entire user journey, from upload to feedback.	Frontend, Backend, Database
Security Testing	Yes	Pass	Checks for unauthorized access and data encryption.	Backend, Database
Scalability Testing	Yes	Pass	Tests system behaviour with an increasing user base.	Backend

5.2 Test Team Details

Role	Name	Responsibilities
Tester	Vansh Gupta	Perform unit, integration, load, and stress testing for reliability.
Developer	Aryan Jolly	Fix issues and optimize backend performance.
Lead QA	Mukund Sarda	Oversee all testing activities, ensure adherence to quality standards.

5.3 Test Schedule

Activity	Start Date	Completion Date	Hours
Requirement Gathering	01/10/2024	02/10/2024	8
Obtain Input Data	03/10/2024	04/10/2024	4
Environment Setup	05/10/2024	06/10/2024	6
Unit Testing	07/10/2024	09/10/2024	12
Integration Testing	10/10/2024	11/10/2024	10
Performance Testing	12/10/2024	13/10/2024	6
Stress Testing	14/10/2024	15/10/2024	6
Load Testing	16/10/2024	17/10/2024	8
Security Testing	18/10/2024	19/10/2024	8
Volume Testing	20/10/2024	21/10/2024	6
User Acceptance Testing	22/10/2024	23/10/2024	10
Bug Fixes and Iterations	24/10/2024	26/10/2024	12
Regression Testing	27/10/2024	28/10/2024	10

5.4 Test Environment

Category	Details
Software Items	Frontend: React.js, Backend: Flask, Database: SQLite
Operating Systems	Backend Server: Linux, Development: Windows/Mac, User Devices: All major OS
Tools	Postman, Selenium, PyTest, JMeter
Cloud Platform	AWS for backend deployment and scalability testing

5.5 Component Decomposition and Type of Testing

S. No	Component	Type of Testing	Technique
1	Frontend (React.js)	Unit, Integration, Load	Black Box
2	Backend (Flask APIs)	Unit, Integration, Stress	White Box
3	Database (SQLite)	Volume, Load	Black Box
4	NLP Models (BERT/BART)	Performance, Accuracy	Tool-Based
5	Quiz Module	Unit, Integration	Black Box

5.6 Test Cases

Test Case ID	Description	Input	Expected Output	Status
T_F1	Validate file upload for supported formats	PDF, Image, DOC	File uploaded successfully.	Pass
T_F2	Handle unsupported file formats	Executable file	Error message: Unsupported file type.	Pass
T_F3	Summarization with BERT	1000-word text	Extractive summary generated.	Pass

T_F4	Summarization with BART	500-word text	Abstractive summary generated.	Pass
T_F5	Quiz generation	Text input	10 MCQs with distractors generated.	Pass
T_BE1	API response under high traffic	Concurrent requests	Responses within acceptable thresholds (<3 sec).	Pass
T_BE2	Database consistency	Simultaneous updates	Data remains consistent and correct.	Pass
T_D1	Handle malformed database queries	Invalid SQL command	Error message returned without crashes.	Pass
T_D2	Validate large file upload (volume testing)	50MB PDF	File processed and stored successfully.	Pass
T_F6	Handle quiz submission and feedback generation	Quiz answers	Correct and incorrect answers highlighted.	Pass

5.7 Error and Exception Handling

Test Case ID	Test Case Description	Debugging Technique
T_F3	Summarization with incorrect output	Fine-tuned NLP model
T_BE1	API crashes under high traffic	Middleware optimization
T_D1	Database inconsistency errors	Adjusted indexing and locking
T_F6	Incorrect quiz results	Refactored question generation logic

5.8) LIMITATIONS OF THE SOLUTION

1. **Technical Limitations**

1.1 **Dependency on Pre-Trained Models**

- **Description**: The summarization and quiz generation functionalities rely heavily on pre-trained NLP models such as BERT and BART. These models, while highly advanced, have inherent limitations: **Input Token Limit**: Both BERT and BART have a maximum input token limit (512 tokens), making it challenging to process large documents without splitting them into smaller chunks. **Generalization Issues**: These models may not perform optimally on domain-specific or highly technical content, such as medical or legal texts, without additional fine-tuning.
- **Impact**: Reduced accuracy or relevance of summaries and quiz questions for niche subject areas. Increased processing time due to chunking and recombining large documents.

1.2 **OCR Performance**

- **Description**: The system uses OCR tools (e.g., EasyOCR, DocTR) to extract text from images or scanned documents. However: Low-resolution or poor-quality images may result in inaccurate text extraction. Handwritten notes, especially with cursive writing, are difficult to process with high precision.
- **Impact**: Errors in extracted text lead to inaccurate summaries and irrelevant quiz questions.

1.3 **High Computational Requirements**

- **Description**: Summarization and quiz generation involve computationally intensive tasks, particularly when using deep learning models. This can lead to: Increased latency for large documents or concurrent user requests. Higher costs for hosting and maintaining the required computational infrastructure.
- **Impact**: Reduced user experience due to delays in processing. Scalability challenges for a large user base.

1.4 **Limited Multilingual Support**

- **Description**: The current implementation primarily supports English-language content. While some OCR and NLP models can process other languages, their accuracy is often lower.
- **Impact**: Excludes users who rely on non-English content.

2. **Operational Limitations**

2.1 **Limited Customization for Users**

- **Description**: While the platform provides options for summary type (extractive/abstractive) and length (short/medium/long), deeper customization is limited.
- **Impact**: Users may not be able to specify additional preferences, such as focusing on specific sections of content or customizing quiz formats.

2.2 **Dependency on Internet Connectivity**

- **Description**: The system is designed as a web application, requiring stable internet connectivity for accessing services like file upload, summarization, and quiz generation.
- **Impact**: Users in regions with limited or unreliable internet access may face difficulties using the platform.

2.3 **Manual Intervention for Edge Cases**

- **Description**: In certain cases, such as extremely noisy images or overly complex documents, manual intervention may be required to refine inputs or outputs.
- **Impact**: Reduces the level of automation promised by the system.

3. **User Experience Limitations**

3.1 **Learning Curve**

- **Description**: Although the platform is designed to be user-friendly, first-time users may find certain features, such as quiz customization or summary length selection, slightly overwhelming.
- **Impact**: Reduced initial adoption rate or user satisfaction.
- **Mitigation**: Onboarding tutorials, tooltips, and a guided walkthrough could be added to improve the user experience.

3.2 **Limited Accessibility Features**

- **Description**: The platform does not currently include extensive accessibility features, such as text-to-speech for visually impaired users or simplified layouts for users with cognitive disabilities.
- **Impact**: Limited inclusivity for users with disabilities.
- **Mitigation**: Future versions could incorporate accessibility standards like WCAG 2.1.

4. **Quiz Generation Limitations**

4.1 **Contextually Irrelevant Distractors**

- **Description**: The distractors generated for quiz questions rely on semantic similarity (e.g., Word2Vec), which may sometimes produce distractors that are too easy or contextually irrelevant.
- **Impact**: Reduces the effectiveness of quizzes as a tool for assessment.
- **Mitigation**: Advanced techniques, such as reinforcement learning-based distractor generation, could improve relevance.

4.2 **Lack of Adaptive Quizzes**

- **Description**: The platform does not yet provide quizzes that adapt dynamically based on user performance.
- **Impact**: Missed opportunities to tailor quizzes to individual learning needs.
- **Mitigation**: Incorporating adaptive learning algorithms could enhance the personalization of quizzes.

CHAPTER- 6

FINDINGS, CONCLUSION, AND FUTURE WORK

6.1) FINDINGS

The **Kruxx** project has demonstrated significant value in streamlining the exam preparation process for students and educators. Key findings include:

1. **Efficiency**:
    - Automated summarization and quiz generation reduce the time and effort required for studying and preparing teaching materials.
    - Customization options (e.g., summary length, quiz difficulty) cater to diverse learning preferences.
2. **User Feedback**:
    - Initial testing and feedback indicate high user satisfaction due to the platform's ease of use and comprehensive feature set.
    - Users appreciate the support for multimedia inputs (e.g., images, videos).
3. **Scalability and Stability**:
    - Load testing results show that the system can handle up to 200 concurrent users with minimal latency.
    - The modular architecture ensures maintainability and future expansion.
4. **Challenges Addressed**:
    - Successfully tackled OCR accuracy issues for low-quality handwritten notes by integrating preprocessing techniques.
    - Optimized NLP models for processing large documents using chunking and batching methods.
5. **Competitive Advantage**:
    - Compared to existing systems, **Kruxx** offers a unique combination of summarization, quiz generation, and analytics in a single platform.

6.2) CONCLUSION

The **Kruxx** project achieves its objective of providing a robust, scalable, and user-friendly solution for summarizing study materials and generating personalized quizzes. By leveraging cutting-edge technologies like **BERT**, **BART**, and **Word2Vec**, the platform addresses the pain points of exam preparation efficiently. Its ability to process diverse input formats and provide actionable feedback sets it apart from existing solutions.

Key achievements include:

- Automated summarization and quiz generation.
- Support for multimedia inputs.
- Detailed performance analytics with personalized feedback.
- High user satisfaction rates during testing phases.

Kruxx is well-suited for students, educators, and institutions looking to optimize the learning process and improve outcomes.

6.3) FUTURE WORK

While the current version of **Kruxx** is robust and feature-rich, there are opportunities for further enhancement:

1. **Multilingual Support**

- Expand summarization and quiz generation capabilities to support multiple languages, catering to global audiences.

2. **Adaptive Learning**

- Integrate AI-based adaptive learning techniques to tailor quizzes and feedback based on user performance and learning pace.

3. **Real-Time Collaboration**

- Enable real-time collaboration features for group study sessions or educator-student interactions.

4. **Advanced Analytics**

- Add predictive analytics to identify learning patterns and suggest study schedules or resources.

5. **Mobile App Development**

- Develop dedicated mobile applications for Android and iOS to improve accessibility and convenience.

6. **Institutional Integration**

- Offer API integrations with learning management systems (LMS) like Moodle, Blackboard, and Google Classroom for wider adoption in educational institutions.

7. **Gamification**

- Introduce gamification elements like badges, leaderboards, and rewards to boost user engagement.

8. **Cloud Deployment**

- Transition to cloud-based architecture (e.g., AWS, Azure) for enhanced scalability and performance.

REFERENCES

1. https://flask.palletsprojects.com
2. https://reactjs.org
3. https://huggingface.co/transformers
4. https://github.com/JaidedAI/EasyOCR
5. https://mindee.github.io/doctr
6. https://github.com/google-research/bert
7. https://code.google.com/archive/p/word2vec/
8. https://www.postman.com
9. https://www.zaproxy.org
10. https://commoncrawl.org
11. https://getbootstrap.com
12. https://towardsdatascience.com
13. https://medium.com/topic/nlp
14. https://scholar.google.com/
15. https://medium.com/
16. https://stackoverflow.com/

BIO DATA (RESUME)

MUKUND SARDA

+919953737055 New Delhi, India
mukundsarda31@gmail.com
linkedin.com/in/mukundsarda
github.com/mukundsarda

EDUCATION

- Bachelor of Technology (B Tech)

Jaypee Institute of Information Technology

July 2021 – June 2025 Noida, India

- **Course** : Computer Science
- **CGPA** : 7.9

- AISSCE (CBSE)

St. Mark's Sr. Sec. Public School

April 2018 – May 2021 New Delhi, India

- **Class 12** : 97% (Science (PCM) with Computer Science)
- **Class 10** : 93.2%

EXPERIENCE

- Open Source Developer Community JIIT

OSDC Core Team Member

July 2022 – June 2023 Noida, India

- Conducted 10+ worshops, talks & learning sessions, educating 200+ attendees about open-source practices & their benefits
- Planned and organized 3+ events & 2 competitions, engaging 300+ participants

- Lawyer24x

Intern

May 2024 – July 2024 New Delhi, India

- Assisted in bug fixing & conducted code reviews, improving code quality & enhancing application stability by 30%
- Enhanced application reliability & security by 40% through integrating security & code analysis plugins in CI/CD pipeline

ACHIEVEMENTS

- Secured 2nd position among 100+ participants in OSDCTF'23 conducted by OSDC JIIT
  Certificate Link
- Achieved top 30 ranking among 90+ teams in Cyber Security Hackathon by EY and Data Security Council of India
  Certificate Link

CERTIFICATIONS

- KodeKloud: Fundamentals of DevOps
  Certificate Link
- Forage: Mastercard Cybersecurity Job Simulation
  Certificate Link
- KodeKloud: Python Basics
  Certificate Link
- CodSoft: Web Development Training
  Certificate Link

SKILLS

- **Languages:** C, C++, Python, HTML, CSS, JavaScript, PHP
- **Libraries/Frameworks:** ReactJs, NodeJs, NumPy, Matplotlib
- **Databases:** MySQl, MongoDB, XAMPP
- **DevOps:** Git/Github, Docker, Kubernetes, Jenkins
- **Tools/Platforms:** VS Code, Jupyter, Excel, PowerBI, Linux
- **Core Concepts:** DSA, OS, Computer Networks, Cybersecurity
- **Soft Skills:** Problem-Solving, Teamwork, Communication

PROJECTS

- Tetris

github.com/mukundsarda/Tetris

- **Deploying & automating 2 versions of Dockerized Tetris** simultaneously with scaling using ArgoCD, Terraform, Jenkins, Kubernetes- increasing availablity & reliablity by 50%
- Implemented vulnerability scanning using tools like Trivy, Sonarqube, OWASP, enhancing security and reducing bugs by 60% and done performance testing using Locust Software

- Grocigo

github.com/mukundsarda/Grocigo

- **Shop management web application** for both shopkeepers & customers with basis 4 CRUD & other useful operations
- Enhanced user engagement by 35% through responsive & interactive frontend built with ReactJS, HTML, CSS, JavaScript, and robust backend using MySQL, PHP, XAMPP

- DMRF

github.com/mukundsarda/DMRF-2.0

- **Delhi Metro Route Finder**- tells shortest route, average fare, average time, no of stations, changes in metro lines
- Developed in C++ using advanced data structures (graphs, vectors), file handling, algorithms (Dijkstra's) to optimize route planning for faster response & enhancing accuracy

- AirQ

github.com/mukundsarda/AirQ

- **AirQ** is a python based project for comprehensive air quality monitoring and prediction, giving 85% accurate results
- Utilizes 3 regression models: linear, random forest, ridge for data processing & finding trends, have data visualization tools, also uses SARIMA modeling for accurate time series analysis, forecasting AQI levels, ANOVA tests to compare AQI levels

- Dockify

github.com/mukundsarda/Dockify

- **Deployed '2048' game using JS, React** on Docker, Kubernetes with Jenkins CI/CD pipelines, improving scalablity by 65%
- Applied 3 security plugins (OWASP, Trivy, Sonarqube) for minimizing vulnerablities & tested performance using Locust

Aryan Jolly

Student/Coder/Graphic Designer

Quick to adapt with a never-give-up attitude

jollyaryan7@gmail.com

8447793947

New Delhi, India

github.com/JollyRocketts

SKILLS

Python | Java | C++ | C | Adobe Photoshop | Adobe Premiere Pro | MySQL | MongoDB | HTML | JavaScript

LANGUAGES

English
Full Professional Proficiency

Hindi
Native or Bilingual Proficiency

INTERESTS

- Coding
- Graphic Designing
- Video Editing
- Game Developing

EDUCATION

B.Tech Computer Science Engineering
Jaypee Institute of Information Technology, Noida Sector-62
09/2021 - Present — *8.2 CGPA*

12th Grade
Apeejay School, Saket
03/2020 - 03/2021 — *96.2%*

PERSONAL PROJECTS

Chords
- Web App that serves as a Social Media for Users with Similar Music Taste using SpotifyAPI, MERN Stack, Python, Flask

2-Player 2D Board Game
- Game wherein both players face-off to gain the highest score possible before the timer runs out. Made using Java, Swing, JPanel, JFrame, ActionListener, Graphics

Silver Link
- One stop application for job opportunities for retired elderly people using MERN Stack, Typescript, LinkedIn API

Blood Bank Management System
- Management system which stores and modifies data regarding donations, requests, blood type and components stock in an SQL database using Python and SQLite3

EXPERIENCE

Graphic Designer for Om Rubber Industries (01/2022 - Present)
Designed logo, t-shirt design, sticker and multiple posters as well as edited a video for the company

Internship at Newgen Software Technology Limited (06/2024 - 07/2024)
Worked on 2 projects using Java, Swing, JFrame, JPanel, ActionListener and Graphics

Graphic Designer for Jaypee Youth Club (Main Hub of JIIT) (03/2023 - 01/2024)
Designed multiple posters for the JIIT Fest, the event brochure and posters for cultural events

Data Science Internship under Acmegrade (07/2022 - 09/2022)
2-month Internship working on a Project analyzing and predicting Stock Prices using Python, Certificate of completion by Acmegrade & MOODI IIT Bombay

ACHIEVEMENTS

Placed 2nd in a Game Development Competition using Scratch

Placed 3rd in an event in the Hackathon organized by Open Source Developers Community Hub of JIIT

CERTIFICATES

Back-End Web Development (09/2022)
Issued by Microsoft Student Learn Ambassador, Google Developer Student Clubs and Devtown

Instagram Clone using HTML and CSS (09/2022)
Issued by Microsoft Student Learn Ambassador, Google Developer Student Clubs and Devtown

JavaScript and React (08/2022)
Issued by Microsoft Student Learn Ambassador, Google Developers Group and Devtown

VANSH GUPTA

Bulandshahr, India

gupta.vansh1203@gmail.com |+91-9634549157| LinkedIn | GitHub

EDUCATION

Jaypee Institute of Information Technology 2021-2025
- Bachelor of Technology- Computer Science Engineering | CGPA-7.6

St. Joseph Sr. Secondary School 2020-2021
- ICSE(CLASSXII) , Percentage-95.8%

St. Joseph Sr. Secondary School 2018-2019
- ICSE(CLASSX), Percentage-92%

TECHNICAL SKILLS

Languages: C, C++, Java
Web Technologies/Framework- Git/GitHub, SpringBoot, Spring, SQLyog, Hibernate
Developer Tools- VS Code, Eclipse IDE, Apache NetBeans
Soft Skills: Communication, Teamwork,Time management

PROJECTS

- **Safar| SpringBoot, Hibernate,HTML,CSS|Source Code**
  - web-based application that enables users to book bus tickets online
  - users can book available buses based on their source and destination cities, reserve the seats, and make payments securely
  - system also provides a UI based dashboard for admin to manage their bus related operations

- **CricInfo|SpringBoot,HTML,CSS |Source Code**
  - An Application that provides users with scorecard of all cricket matches along with players statistics

- **Route Solver| C++|Source Code**
  - A C++ project that helps in managing several ships of a port.
  - Uses various C++ concepts such as linked list, binary tree,binary search trees ,etc.

- **EveryThing| Spring,Java,MySQL| Source Code**
  - An e-commerce platform based on Java and Spring where users can buy products

ACADEMIC ACHIEVEMENTS

- Over 500+ problems solved on Leetcode| Profile Link
- Over 200+ problems solved on various Data Structures and Algorithms| Profile Link
- Over 150+ problems solved on CodeForces | Profile Link
- Participated in hackathons like GDSC Hackin Summer'22 organised by college

INTERESTS

- E-Sports , World Affairs and Geopolitics